

CSC 480 – Artificial Intelligence I
2025-26 – Spring Term
Instructor: Kenny Davila Castellanos

Homework #1: Solving Problems by Searching

Objective

The first goal of this homework is to implement different search algorithms studied in class. The second goal is to evaluate these algorithms on a practical application based on a real-life use case.

Summary

In this assignment, you are tasked with implementing breadth-first search, uniform cost search and A* search. You will use these algorithms to optimize the routes used to make multiple deliveries for a fictional company. The algorithms will help the company find the optimal routes for drivers that need to deliver multiple packages per trip. Some initial Python code will be provided, and you must adhere to the structure of this code. After implementing all required algorithms and functions, your ultimate goal is to carry out multiple comparisons between different configurations for your program, and then prepare a written report with your findings. A detailed explanation of the requirements is provided in the following sections.

Problem Description

A foreign company has developed an online store app. They sell food and other fresh products that need to be delivered within a short period of time. So far, the company groups these online orders into small batches, where each of these batches might include delivery locations which are very far away from each other. The batches are then assigned to specific delivery drivers who then need to figure out the delivery routes on their own. As the company grows, this whole process has become too complex, and quite often the drivers do not pick good routes for delivery. This in turns translates to significant delays and even losses due to products arriving too late.

Given your expertise on the topic, the company hired you to develop a simple program that uses search algorithms to help the drivers figure out the routes to deliver everything as fast as possible. This program takes as input two files: a graph that represent the city, and a set of search requests (test cases). The program loads the files, creates a representation of the problem, and runs different search algorithms: Breadth-first Search, Uniform Cost Search, and A*. The program then collects the solution produced by each algorithm, and will print it along with other auxiliary statistics such as the number of nodes reached, the number of nodes explored, and the total search time.

City Graph. Search problems often can be represented at different levels of granularity, effectively allowing us to balance the level of detail of the plan vs the time required to generate the plan. Here, we

are not interested in providing low-level navigation instructions for the drivers. Instead, we will handle high-level regions of the city, and we will use these as the locations that the driver must visit to make one or multiple deliveries. The idea is to create a high-level plan indicating the order in which these regions or locations must be visited to minimize the total distance. The driver can in turn, figure out how to best deliver one or multiple items within each location. In this case, we start with the city of Tegucigalpa as the pilot for this project. The city is **initially divided in 24 regions** akin to the neighborhoods in Chicago. Instead of using their long real names, we will be using letters from “A” to “X”. Then, each of these 24 regions is further subdivided into 4 sub-regions (e.g., “A1”, “A2”, “A3”, “A4”) producing a total of 96 locations in the map. This idea is illustrated in **Figure 1**.

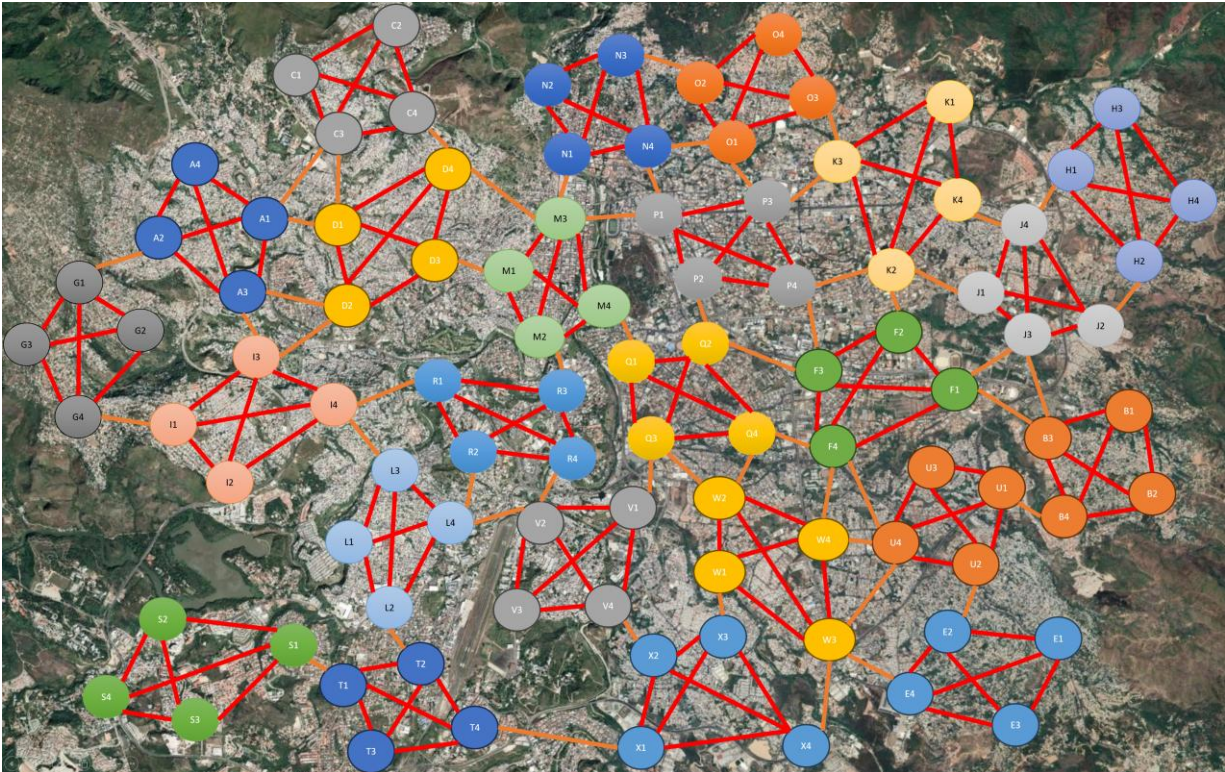


Figure 1. A simplified graph representing the city of Tegucigalpa.

The graph in Figure 1 also shows the connections between different locations. Note that not every pair of nodes that could be connected is connected. **We mainly consider connections created by using main roads. It is also assumed that all locations within a single given region (e.g., “A1” to “A4”, “B1” to “B4”, etc.) can be easily reached from each other, so they form small *cliques* within the city graph.** Another important factor here is the geography which often leads to an absence of a connection between locations that are otherwise very close to each other. All information related to this map is included within the file “tegucigalpa.json”. An illustration of this map can be found on the file “city_graph.svg”

Search Request. **We will call every test case a search request**, which is a simple grouping of the minimal information required to execute a search in this map. Each search request has a name that can be used to refer to specific test cases in the final analysis. **Every search begins from a *starting location*, and must visit a *list of target locations* (deliveries), and then return to the starting location** (e.g., the driver brings back the vehicle). The search process needs to find a route that **minimizes the total traveled distance (in miles).**

Implementation Details

For this work, you will be given a complete project in Python with multiple classes for specific tasks. To keep things simple, you are given a significant amount of **boiler plate** code. This includes the classes and functions used deal with the input data. Feel free to read and study this code, but **do not make any changes or you might receive significant scoring penalties**. Your main task is to implement specific functions directly related to the **problem formulation** and the **implementation of the search algorithms** themselves. **Table 1** summarizes the code and data files given to you.

Name	Type	Description	Modify
Package: AI1_HW_Problem			
<code>__init__.py</code>	Code	Python module file.	No
<code>search_request.py</code>	Code	Defines the SearchRequest class.	No
<code>map.py</code>	Code	Defines the CityMap class	No
<code>state.py</code>	Code	Defines the State class.	No
<code>problem.py</code>	Code	Defines the Problem class.	Yes
Package: AI1_HW1_Search			
<code>__init__.py</code>	Code	Python module file.	No
<code>search_tree_node.py</code>	Code	Defines the SearchTreeNode class.	No
<code>search_algorithms.py</code>	Code	Defines the SearchAlgorithms class.	Yes
<code>search_results.py</code>	Code	Defines the SearchResults class.	No
Root Directory			
<code>main.py</code>	Code	Entry point for the program. It puts everything together.	Maybe
<code>tegucigalpa.json</code>	Data	Simplified map of the city of Tegucigalpa (Honduras). All provided test cases are based on this map. Do not modify.	No
<code>city_graph.svg</code>	Data	A visualization of the graph represented in the map file.	No
<code>test_cases.json</code>	Data	Examples of search requests (Test Cases) in JSON format. Feel free to edit this file to add your own custom test cases for debugging.	Maybe

Table 1. Summary of files included in the code project.

SearchRequest. This class is used to represent a Search (a test case) that needs to be executed. As described before, each search request has a name (`__name`) used to identify individual test cases. Each search begins from one specific location (`__start`), and requires visiting a list of targets (`__targets`). The class provides a convenient static method (`FromTestCasesFile`) for loading multiple test cases from a given JSON file. **Do not change anything on this file.**

CityMap. This class is used to represent the whole city map. It includes the name of the city (`__name`), a list of all unique locations (`__locations`), a dictionary of all connections with their corresponding costs (`__connections`). In addition, it also includes the 2D map positions representing the center of each location (`__map_positions`), and a factor (`__map_scale`) used to convert the Euclidean distance between any pair of locations to a value in miles. During A* search, heuristics can be generated using the straight-line distance between any pair of locations in the map, and the class provides a method for this. However, since the same distances are likely to be computed over and over again for during search, this class uses an additional attribute to cache the values of all pair-wise distances (`__pairwise_distance_cache`). This cache is populated in the constructor and allows for quicker retrieval of the estimated distance between

heuristic

two locations. The class provides a static method (**FromFile**) for loading a city map from a JSON file. **Do not change anything on this file.**

State. This is a problem-specific class used to represent an individual **state** (node) of the **State Space** (graph). In this application, the state class includes the **current location** of the driver (**__current_loc**), the **status of each target** (**__visited_targets**), and the **atomic representation** of the state (**__atomic**). Current location is a String (map location name). For a given search request, a list of target locations is provided, and the values in the **__visited_targets** attribute must be aligned with these targets, indicating if the corresponding target has already been visited (True) or not (False). For example, if a search requires visiting locations ['F2', 'A2', 'K3', 'X4', 'D2'], and **__visited_targets** is equals to [True, False, False, True, False], this would indicate that locations 'F2' and 'X4' have been visited already, and that 'A2', 'K3' and 'D2' are still pending. **Note that the locations in the Search Request object are NOT sorted (this is completely intentional).** These locations simply follow the order used in the test case file. This order should not have any effect on the search process. Now, the number of locations to visit directly changes the size of the state space and will have an impact on performance. Finally, the **atomic** attribute keeps an "atomic representation" of the whole state, which is required for keeping records of the nodes that have been reached and explored. **Do not change anything on this file.**

Problem. This class handles most of the problem formulation. **It is also one of the classes where you need to make multiple changes.** For this application in particular, we need two attributes: the city map (**__city_map**) and the search request (**__current_case**). **It is the job of this class to put together the information about the map and the search request to generate the states, actions, transitions, costs and heuristics needed for search.** The functions that **you must implement** are described in **Table 2**.

Method	Description
get_initial_state() -> State	Uses the information from the map (__city_map) and the search request (__current_case) to generate the Initial State.
is_goal_state(state: State) -> bool	Checks if the given state object represent a goal state. This can be determined using information stored in the state itself and also in the __current_case object.
generate_children(state: State) -> List[State]	Given a state, this function generates a List of the Children states. This function basically implements the transition model. First, you need to consider which locations are neighbors of current location in the given state (the city map can help with that). Each action here represents traveling to one of these neighbors. For each of these actions, you will have to generate a new state object. Keep in mind that some of these locations might be part of the targets. This means that you might need to update the vector of visited nodes for the child state accordingly.
get_action_cost(state: State, action: str) -> float	Returns the cost of executing the given action on the given state. Here it is equal to the distance in miles from current location to new location. This can be determined using the map.
estimate_cost_to_solution(state: State) -> float	This is the Heuristic function used by the A* search. This function estimates the cost of reaching a goal state from the given state.

Table 2. Methods to Implement for the Problem class.

SearchTreeNode. This class represents the individual **nodes** of a **Search Tree**. Every node is linked to its parent (**__parent**). Each node is linked to a given unique State (**__state**) from the State Space. It also stores the action (**__action**) used to transition from the state at parent node to the state in current node. Then, the node stores the total cost (in miles) of the path from the root to the itself (**__path_cost**). We do not need to connect the nodes to their children because the tree is constructed in a top-down fashion, keeping references to all leaves (e.g., the frontier). When a goal state is finally extracted from the frontier, we simply navigate the path from that node back to the root of the tree to reconstruct the action sequence (e.g., the solution). Note that this class is generic, all problem-specific data is stored in the linked state, not on the SearchTreeNode directly. **Do not change anything on this file.**

The SearchTreeNode class has an id attribute (**__node_id**). This is a simple correlative that counts how many nodes have been created so far. It is used as the default key for sorting elements of this class. That is, this is the attribute used to determine if a given node is "smaller" than another in the "**__lt__**" method which is used when two objects of this class are compared against each other ("**a < b**"). However, SearchTreeNodes should be sorted externally because the frontier uses a priority queue, with priority values that changes depending on the search algorithm used. Two nodes might still be tied on the main priority value, and in these cases the tie will be broken by **__node_id**.

SearchAlgorithms. This class implements different search algorithms. **It is also one of the classes where you need to make multiple changes.** As shown in Table 3, you will be implementing: **breadth-first search**, **uniform cost search**, and the **A* search**. For all three search functions, **the only input required is the problem, which includes a reference to current CityMap and current Search Request.** Each function then needs to compute and store search results and other statistics inside of a SearchResults object. The class includes some **boiler plate code which you must not change.**

Method	Description
breadth_first_search(problem: Problem) -> SearchResults	Implementation of the Breadth-first Search (BFS) algorithm.
uniform_cost_search(problem: Problem) -> SearchResults	Implementation of the Uniform Cost Search (UCS) algorithm.
A_star_search(problem: Problem) -> SearchResults	Implementation of the A* Search algorithm.

Table 3. Methods to Implement for the SearchAlgorithms class.

SearchResults. This class collects and groups all relevant results of any given search. It has a list with the solution path (**__solution_path**), and the corresponding cost of the solution (**__solution_cost**). In case of failure, both of these attributes will be None. To compare different search algorithms, it also stores the counts of nodes reached (**__nodes_reached**) and nodes explored (**__nodes_explored**). **Do not change anything on this file.**

Main. This is the class that puts everything together. Technically speaking, you are giving some boiler plate code that does everything you need here. **You might not need to make any changes to this file, but feel free to change things as you need for debugging, but the final version should still load the given map and test cases, and should run all 3 search algorithms on each test case.**

REREAD!!!

Estimating the Cost of Reaching a Goal State: The Heuristic

One of the main components of the A* search algorithm is the heuristic function which is used to estimate the cost of reaching a goal state from state in the search space. As it was explained in class, this function must not over-estimate the real cost of reaching the goal from any state. At the same time, a good heuristic should strive to make an accurate prediction, getting as close as possible to the real cost without taking too much time on the process. In this assignment, you will have to implement a simple yet very effective heuristic. This estimation is based on the following observations:

1. When there are no deliveries left, the only cost to consider is the cost of traveling from current location back to the starting location to complete the trip.
2. When there is only one delivery left, the costs to consider are traveling from current location to the pending target location, and then traveling from that location back to the starting location.
3. When there are two or more deliveries left, the costs to consider is traveling from current location to the next delivery location, and then going from there to all other delivery locations. Afterwards, there is the cost of going from the last delivery location back to the starting location.

Based on these observations, we can design a heuristic function by using a relaxed version of the delivery optimization problem. **Observation #1** can be used to estimate the cost when there are no deliveries left. For this, we can use the **straight-line distance (SL_Dist)** between the **current location ($L_{current}$)** and the **final location (L_{final})**. Remember that the final location is the same as the starting location of the search. This idea is exactly like the one used in the navigation example on the textbook. The resulting distance is not accurate but it gives us a lower-bound of the real cost making it a valid heuristic.

For **Observations #2 and #3**, these can be considered the same. Case #2 is simply a special version of Case #3, with the only difference that traveling to “all other delivery locations” has a cost of 0. In general, we can use the same straight-line distance to estimate the cost of each part of the trip (beginning, middle, final). However, we do not know the optimal order for making all deliveries, and that is exactly what we want the algorithm to figure out in the first place. Still, to determine a lower-bound cost, we do not need to know the exact order, we can make a coarse estimation based on the “best case” scenario, and we know that the real cost is guaranteed to be same or higher, thus making this idea a valid heuristic.

To make an estimation of the lower-bound cost for our best-case scenario, we need to consider **current location ($L_{current}$)** and the **final location (L_{final})**. Remember that the final location is the same as the starting location of the search. In addition, we will need to determine **two additional key locations: the pending delivery target that is the closest to the final location (L_{close})**, and the **pending delivery target that is the farthest from the final location (L_{far})**. The optimal path has to visit all these locations at some point. We simply do not know the optimal order for L_{close} or L_{far} , so the optimal path could be either $[L_{current}, \dots, L_{close}, \dots, L_{far}, \dots, L_{final}]$ or $[L_{current}, \dots, L_{far}, \dots, L_{close}, \dots, L_{final}]$. Again, we do not need to worry about making an exact estimation, we only need to determine a **lower-bound** for the overall cost. Regardless of the optimal order, the best path must follow a route that will connect L_{close} to L_{far} at some point (from first to second or backwards). We can use the **straight-line distance (SL_Dist)** to estimate a minimum cost for making this portion of the trip as:

$$C_{middle} = SL_Dist(L_{close}, L_{far})$$

In the best-case scenario, the cost of C_{middle} will also include the cost of traveling to all other delivery locations (e.g., they are conveniently part of the optimal path between L_{close} and L_{far}). For the average case, we would expect this over-simplification to always sub estimate the real cost of the best route. Also, C_{middle} only considers the cost of traveling between L_{close} and L_{far} , and since we do not know what other locations will not be visited along this route, it is hard to estimate the cost of visiting them. However, we still know that the driver needs to travel from $L_{current}$ to either $[L_{close}$ and then reach $L_{far}]$ or $[L_{far}$ and then reach $L_{close}]$. We will make a conservative estimation based on whichever of these two locations is the closest to $L_{current}$. So, the cost of the first part of the trip is estimated as:

$$C_{beginning} = \min(SL_Dist(L_{current}, L_{close}), SL_Dist(L_{current}, L_{far}))$$

Then, the last part of the trip is to go back from either L_{close} or L_{far} to the final position L_{final} . We do not know if any other delivery location will be visited afterwards, but we are considering a base-case scenario here. We must use a conservative estimation based on the closest location which we already know is L_{close} . So, the cost of the final part of the trip is estimated as:

$$C_{final} = SL_Dist(L_{close}, L_{final})$$

Putting these 3 parts together, our estimation of the cost of reaching the goal state from a given current state is: $C_{beginning} + C_{middle} + C_{final}$. Note that when there is exactly one delivery left, this means that $L_{close} = L_{far}$, and then $C_{middle} = 0$. Your task is to implement this formula in the Problem class, in the method for estimation of the cost as follows:

$$H(state) = \begin{cases} SL_Dist(L_{current}, L_{final}), & \text{pending deliveries} = 0 \\ C_{beginning} + C_{middle} + C_{final}, & \text{pending deliveries} \geq 1 \end{cases}$$

The Main Program (70%)

For this assignment, you are given a significant amount of boiler plate code which cannot be changed. Also, the initial code is incomplete but runs and you should gradually make changes ensuring that it still works after making changes. You need to implement all required functions in the SearchAlgorithms and Problem classes. Example of expected outputs for **selected** test cases included in the zip file:

Test Case: Easy_3			
Starting Location:	D1		
Delivery Locations	C2, A2, D3		
	Breadth-First Search	Uniform Cost Search	A* Search
Solution Found	[D1] → A1 → (A2) → A1 → C3 → (C2) → C3 → [D1] → (D3) → [D1]	[D1] → (D3) → D4 → C4 → (C2) → C3 → A1 → (A2) → A1 → [D1]	[D1] → (D3) → D4 → C4 → (C2) → C3 → A1 → (A2) → A1 → [D1]
Solution Cost (mi)	10.9474	10.1424	10.1424
Nodes Reached	420	490	83
Nodes Explored	329	400	40
Search Time (s)	0.001000	0.007510	0.001002

Test Case: hard_1			
Starting Location:	Q2		
Delivery Locations	A2, S3, G2, W2, E2, J3, H2		
	Breadth-First Search	Uniform Cost Search	A* Search
Solution Found	[Q2] → F3 → F1 → (J3) → J2 → (H2) → J2 → J1 → K2 → P4 → P1 → M3 → D4 → D1 → A1 → (A2) → G1 → (G2) → G4 → I1 → I4 → L3 → L2 → T2 → T1 → S1 → (S3) → S1 → T1 → T4 → X1 → X4 → W3 → E4 → (E2) → E4 → W3 → (W2) → Q3 → [Q2]	[Q2] → Q1 → M4 → M1 → D3 → D1 → A1 → (A2) → G1 → (G2) → G4 → I1 → I4 → L3 → L2 → T2 → T1 → S1 → (S3) → S1 → T1 → T4 → X1 → X3 → W1 → (W2) → W4 → U4 → U2 → (E2) → U2 → U1 → B4 → B3 → (J3) → J2 → (H2) → J2 → (J3) → F1 → F3 → [Q2]	[Q2] → Q1 → M4 → M1 → D3 → D1 → A1 → (A2) → G1 → (G2) → G4 → I1 → I4 → L3 → L2 → T2 → T1 → S1 → (S3) → S1 → T1 → T4 → X1 → X3 → W1 → (W2) → W4 → U4 → U2 → (E2) → U2 → U1 → B4 → B3 → (J3) → J2 → (H2) → J2 → (J3) → F1 → F3 → [Q2]
Solution Cost (mi)	48.0053	44.6232	44.6232
Nodes Reached	11778	11721	6864
Nodes Explored	11669	11635	6082
Search Time (s)	0.050047	0.074071	0.059059

Additional Considerations

- 1) Base code is provided for this work. You must respect this code. **Not using the functions and classes provided can lead to severe scoring penalties.**
- 2) Programs that do **not compile will receive a score of 0. No exceptions.**

Data Collection

You have been provided with a file that contains multiple tests cases (search requests). You are required to analyze how different algorithms behave over each of these test cases. It is important to consider the solution itself, the cost and optimality of the solution. You must also consider the search time, and the number of nodes that have reached and further explored. Basically, you must collect the output to create similar tables to the ones shown above (one per test case).

Keep in mind that even with the same inputs, an algorithm might experience a lot of variation in search time. This is because there are multiple programs working at the same time in the computer, and they are all competing for important resources such as CPU time, memory and access to secondary storage. To deal with this variability, **you will need to take multiple measurements (at least 5) and then use the median of these values.** The median is typically far less sensitive to outliers than the mean. Also, to make a fair comparison, all searches should be run on the same computer.

The Report (30%)

You have to prepare a written report with the results for the experiments carried out here. The report should clearly state the name of the author. It must have the following sections:

1. Introduction

- a. Briefly explain the whole assignment in your own words.

2. Code architecture (0.5 – 1.0 pages max)

- a. Briefly describe how your custom code works.
- b. Usage of diagrams is highly encouraged.
- c. Code snippets might be included but you still have to explain what they do with words.

3. Computer Specs.

- a. Provide all the relevant specs for the computer that you used to run your experiments.
- b. Keep in mind that a single computer should be used to collect all data.

4. Results.

- a. For each test case:
 - i. Results table with the requested metrics for all search algorithm.
- b. An in-depth analysis of these results
 - i. Consider the effect of the number of target locations over search time, nodes reached and nodes explored.
 - ii. Consider the effect of the search algorithm used over search time, nodes reached and nodes explored.
 - iii. Anything else that you consider relevant/interesting.
 - iv. Ideally, include any plots that you might see relevant.

5. Difficulties (Optional).

- a. Provide a brief summary of difficulties that you faced while implementing this project.

6. New application (0.25 – 0.5 pages).

- a. Propose an entirely different application where you would use these Search Algorithms.
- b. Describe the application in detail.
- c. Be specific, what is the **problem formulation**?
- d. Be concise. Do not use fillers to reach the minimum length.
- e. Be creative. Do not use chat-GPT, it will be obvious if you do.

7. Feedback (0.25 – 0.5 pages)

- a. What did you think about this assignment?

8. Conclusions.

Delivery Instructions

You are being provided with a Zip file including a basic PyCharm project. You are required to implement your solution using the code provided. When done, **you should simply zip back the entire project and submit as a ZIP file.**

1. **DO NOT rename any internal project files!**
2. **Using other compression formats (.rar) will lead to a penalty.**
3. **If you are using virtual environments in python, DO NOT include it on your zip file! It will make the project huge!**

File names. The zip file that you submit should use “[Last Name(s)], [Given Name(s)].zip” as it appears in D2L. For example, “Kenny Davila Castellanos” (Davila Castellanos is two last names), would have to submit

the homework with the name “Davila Castellanos, Kenny.zip”. Another student named “Kenny Mauricio Davila” (Mauricio is a middle name), would have to submit the homework as “Davila, Kenny Mauricio.zip”.

Policies

1. This homework is meant to be worked **individually**.
2. All general policies about Plagiarism and Cheating apply to this mini-project. If you plagiarize or receive code from other people, you will be caught and you will receive a score of 0, and the academic integrity violation will be filed.
3. There is a data collection component in this assignment. Creating fake numbers is highly unethical and is considered Cheating.
4. Use of AI tools must be restricted to Ethical Usage only. You must not use any code generators for completing this assignment. Doing so will be considered a form of cheating and the submission will become eligible to receive a score of 0.
5. Do not post your solutions online and do not share them with anyone. It is your responsibility to safe guard your private data.
6. Code that does not compile due to syntax and/or semantic errors will automatically receive a score of 0. It is hard to assign partial credit when I cannot even run your code.
7. Must use PyCharm if you want to receive technical support from my end.
8. You must follow the delivery instructions.
9. Late submissions without justification might receive penalties proportional to the number of hours late. See the syllabus for concrete details on late penalties.
10. The code and documentation highlight using specific approaches for certain things. **You are expected to follow these approaches. Any different approach that you use without consulting me first is likely to receive a score of 0.**
11. Do ask for help if anything is unclear, but do it in a timely manner (e.g., by e-mail or during the Office Hours).